8253jang@daum.net
장희정

SpringBoot기반의 Restful API와 React 연결하기

주요기능

- **회원관리(아이디, 비밀번호, 이름, 주소, role, 등록일)**
  - 회원가입(아이디중복체크)
  - 로그인
  - 로그아웃

- **게시판관리(글번호, 제목, 내용, 작성자, 등록일, 수정일)**
  - 게시물 전체조회– 아무나 조회
  - 게시물 상세보기–인증된사용자 상세보기
  - 게시물등록–인증된 사용자 만이 등록가능
  - 게시물수정 - 인증된 사용자 + 자신이 쓴 글인 경우만
  - 게시물 삭제- 인증된 사용자 + 자신이 쓴 글인 경우만

React환경 세팅

1. **프로젝트를 생성한다.**

```
npm create vite@latest step18-client-jwt
```

2. **생성된 프로젝트로 이동한다.**

```
cd .\step18-client-jwt\
```

3. **필요한 lib install한다.**

```
npm i

npm install react-router-dom

npm install axios
```

8253jang@daum.net
장희정

package.json파일확인

```
12    "dependencies": {
13      "axios": "^1.10.0",
14      "react": "^19.1.0",
15      "react-dom": "^19.1.0",
16      "react-router-dom": "^7.6.2"
17    },
```

전체화면구성

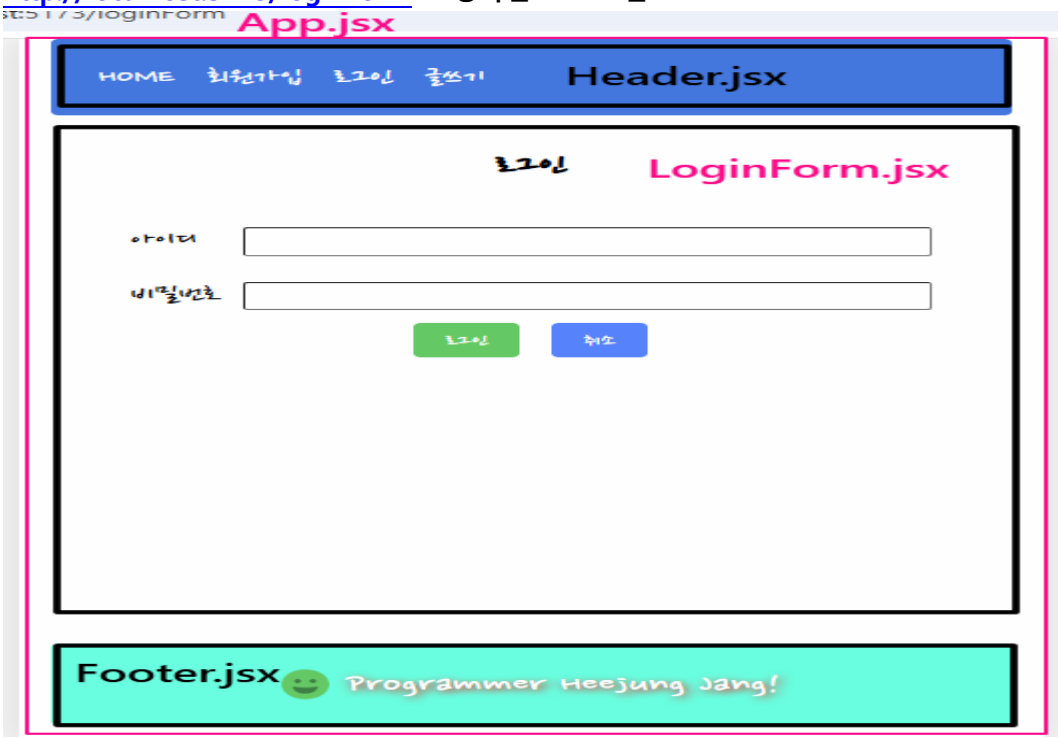
http://localhost:5173/요청 화면

8253jang@daum.net
장희정

http://localhost:5173/joinForm 요청화면 - 회원가입

8253jang@daum.net
장희정

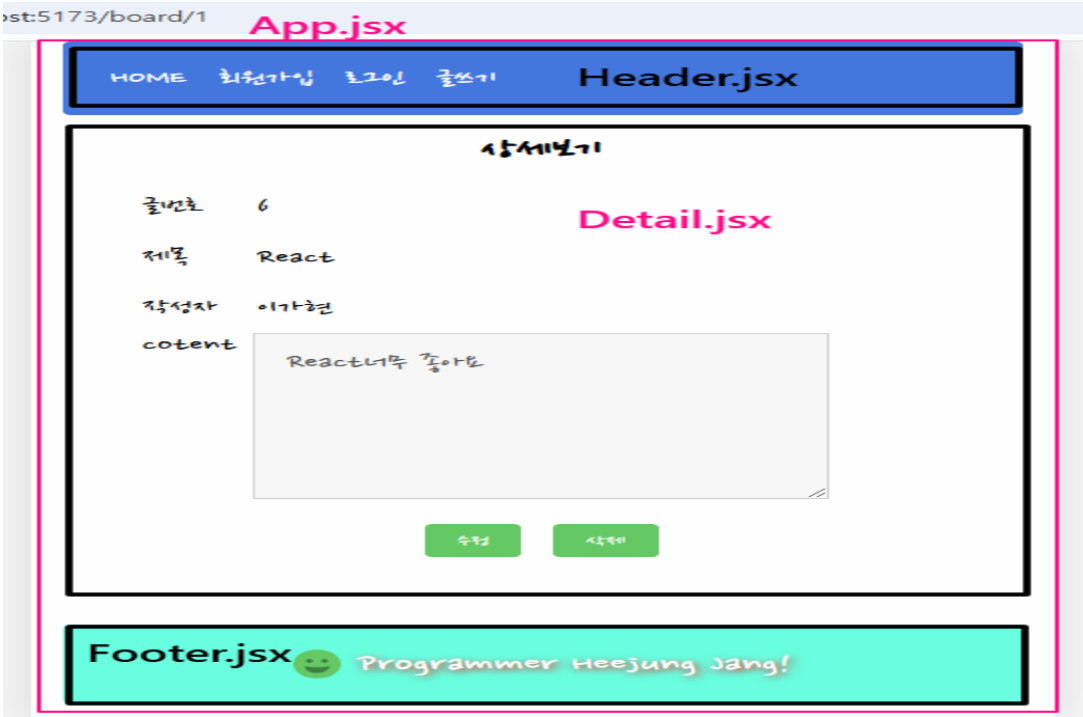
http://localhost:5173/loginForm 요청화면 - 로그인



8253jang@daum.net
장희정

http://localhost:5173/saveForm 요청화면 - 게시물 등록

http://localhost:5173/board/1 요청화면 – 상세페이지



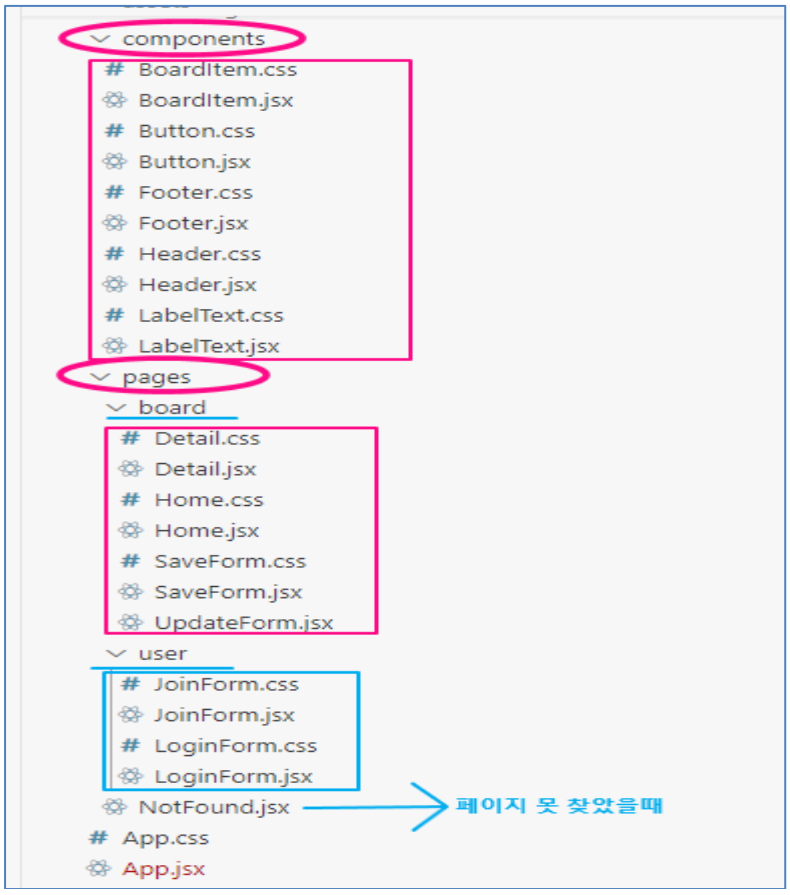
8253jang@daum.net
장희정

http://localhost:5173/updateForm/3 요청화면 - 수정폼

프로젝트 Directory구조

8253jang@daum.net
장희정

8253jang@daum.net
장희정



먼저, 각 화면을 담당하는 컴포넌트 파일들을 만들어 놓는다.

main.jsx에 BrowserRouter를 연결한다.

화면캡쳐	main.jsx 소스
step19-client-jwt-temp > src > main.jsx > router 14 const router = createBrowserRouter([15 { 16 path: '/', 17 element: <App />, // 레이아웃 (Header + Footer + Outlet) 18 errorElement: <NotFound />, // 에러(없는 경로 등) 기본 처리 19 children: [20 {index: true, element: <Home />}, 21 {path: 'saveForm', element: <SaveForm/>,}, 22 {path: 'board/:id', element: <Detail/>,}, 23 {path: 'updateForm/:id', element: <UpdateForm/>,}, 24 {path: 'loginForm', element: <LoginForm/>,}, 25 {path: 'joinForm', element: <JoinForm/>,}, 26 {path: '*', element: <NotFound />,// 나머지 모든 서브 경로 처리 27 }, 28], 29 }, 30]); 31 32 createRoot(document.getElementById('root')).render(33 <RouterProvider router={router} /> 34)	import { createRoot } from 'react-dom/client' import App from './App.jsx' import './index.css' import {RouterProvider, createBrowserRouter} from "react-router-dom" import Home from './pages/board/Home.jsx' import NotFound from './pages/NotFound.jsx' import SaveForm from './pages/board/SaveForm.jsx' import Detail from './pages/board/Detail.jsx' import UpdateForm from './pages/board/UpdateForm.jsx' import LoginForm from './pages/user/LoginForm.jsx' import JoinForm from './pages/user/JoinForm.jsx' const router = createBrowserRouter([{ path: '/', element: <App />, // 레이아웃 (Header + Footer +

	Outlet) errorElement: <NotFound />, // 에러(없는 경로 등) 기본 처리 children: [{index: true, element: <Home />}, {path: 'saveForm', element: <SaveForm/>,}, {path: 'board/:id', element: <Detail/>,}, {path: 'updateForm/:id', element: <UpdateForm/>,}, {path: 'loginForm', element: <LoginForm/>,}, {path: 'joinForm', element: <JoinForm/>,}, {path: '*', element: <NotFound />,// 나머지 모든 서브 경로 처리 },], },]); createRoot(document.getElementById('root')).render(<RouterProvider router={router} />)

App.jsx피일에 Layout을 정한다.

화면캡쳐	App.jsx 소스

2 import './App.css'
3 import { Outlet } from 'react-router-dom'
4 import Header from './components/Header'
5 import Footer from './components/Footer'
6
7 function App() {
8 return (
9 >
10 <Header/>
11 <main className="AppManin">>
12 <Outlet/>
13 </main>
14 <Footer />
15
16)
17 }
18
19 export default App
	import './App.css'
	import { Outlet } from 'react-router-dom'
	import Header from './components/Header'
	import Footer from './components/Footer'
	
	function App() {
	return (
	 >
	<Header/>
	<main class="AppManin">>
	<Outlet/>
	</main>
	<Footer />
	
	)
	}
	export default App

Route 세팅이 완료 된 후에 요청주소로 접속하여 페이지가 잘 열리지는 확인한다.

예) http://localhost:5173/ - Home 컴포넌트

예) http://localhost:5173/saveForm - SaveForm 컴포넌트

예) http://localhost:5173/board/1 - Detail 컴포넌트

예) http://localhost:5173/updateForm/1 - UpdateForm 컴포넌트
예) http://localhost:5173/loginForm - LoginForm 컴포넌트
예) http://localhost:5173/joinForm - JoinForm 컴포넌트

-회원관리 화면 구성

1)회원 가입

user > JoinForm.jsx | JoinForm.css

2) 로그인

user > LoginForm.jsx |LoginForm.css

-게시판관리 화면구성

1) 게시물 등록

board > SaveForm.jsx | SaveForm.css

2) 전체게시물

board > Home.jsx | Home.css

3) 상세보기

board > Detail.jsx | Detail.css

4) 수정하기

board > UpdateForm.jsx | UpdateForm.css

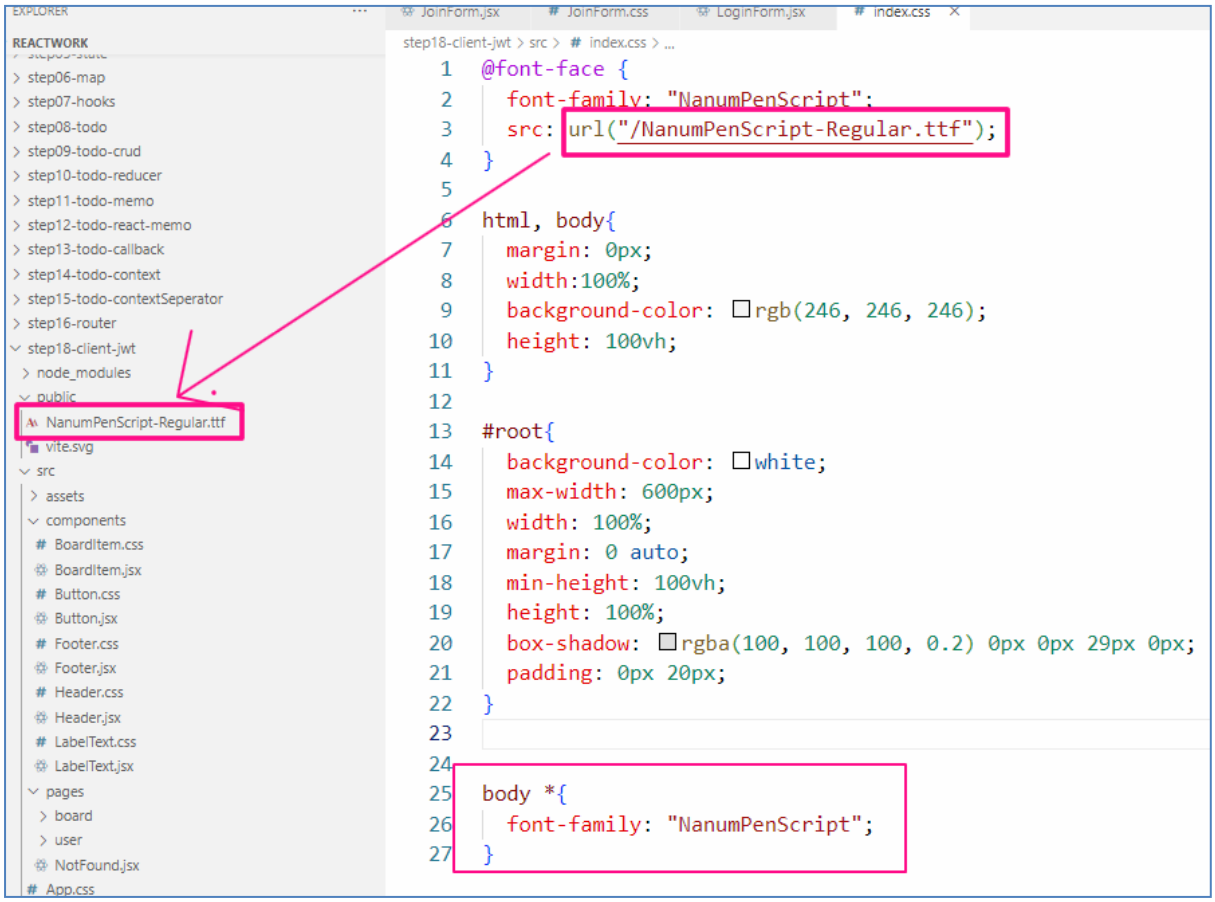
화면구성 만들어보자!

전체 페이지에 공통으로 적용 할 CSS

src > index.css | App.css

index.css파일

8253jang@daum.net
장희정



Index.css

```
@font-face {
  font-family: "NanumPenScript";
  src: url("/NanumPenScript-Regular.ttf");
}

html, body{
  margin: 0px;
  width:100%;
  background-color: rgb(246, 246, 246);
  height: 100vh;
}

#root{
  background-color: white;
  max-width: 600px;
  width: 100%;
  margin: 0 auto;
  min-height: 100vh;
  height: 100%;
  box-shadow: rgba(100, 100, 100, 0.2) 0px 0px 29px 0px;
  padding: 0px 20px;
```

```
}

body *{
  font-family: "NanumPenScript";
}
```

App.css 파일

tep18-client-jwt > src > # App.css > ...

```
.App {
  display: grid;
  grid-template-rows: auto 1fr auto; /* Header, content, Footer */
  height: 100vh; /* 전체 화면 높이 */
}

 .AppManin{
  overflow: auto;
}
```

App.css

```
.App {
  display: grid;
  grid-template-rows: auto 1fr auto; /* Header, content, Footer */
  height: 100vh; /* 전체 화면 높이 */
}

 .AppManin{
  overflow: auto;
}
```

src > components > Header.jsx | Header.css | Footer.jsx | Footer.css

Header.jsx파일

8253jang@daum.net
장희정

```
ep18-client-jwt > src > components > Header.jsx > ...
import { Link } from 'react-router-dom';
import './Header.css';

const Header =()=>{
    return (
      <>
        <div class="Header">
            <Link to="/" >HOME</Link>
            <Link to="/joinForm">회원가입</Link>
            <Link to="/loginForm">로그인</Link>
            <Link to="/saveForm">글쓰기</Link>
        </div>
      </>
    )
  }

  export default Header;
```

Header.jsx

```
import { Link } from 'react-router-dom';
import './Header.css';

const Header =()=>{
    return (
      <>
        <div class="Header">
            <Link to="/" >HOME</Link>
            <Link to="/joinForm">회원가입</Link>
            <Link to="/loginForm">로그인</Link>
            <Link to="/saveForm">글쓰기</Link>
        </div>
      </>
    )
  }

  export default Header;
```

Header.css파일

8253jang@daum.net
장희정

step18-client-jwt > src > components > # Header.css > .Header

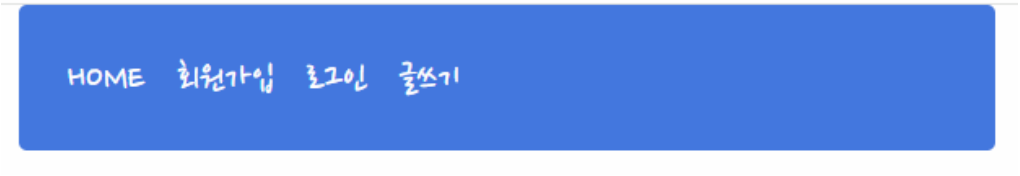
```
.Header{
    display: flex;
    gap: 20px;
    justify-content: flex-start;
    padding: 30px;
    background-color: rgb(67, 119, 223);
    font-size: 25px;
    color: white;
    border-radius: 5px;
}

a{
    text-decoration: none;
    color: white;
}

a:hover{
    text-decoration: underline;
    color: pink;
}
```

Header.css

```
.Header{
    display: flex;
    gap: 20px;
    justify-content: flex-start;
    padding: 30px;
    background-color: rgb(67, 119, 223);
    font-size: 25px;
    color: white;
    border-radius: 5px;
}

a{
    text-decoration: none;
    color: white;
}

a:hover{
    text-decoration: underline;
    color: pink;
}
```

8253jang@daum.net
장희정

실행화면



Footer.jsx파일

```
tep18-client-jwt > src > components > Footer.jsx > ...
1  import "./Footer.css";
2  import emotion1 from './../assets/emotion1.png';
3  const Footer =()=>{
4      return (
5        <>
6          <div className="Footer">
7            <img src={emotion1} />
8            <div className="Footer_div">Programmer Heejung Jang!</div>
9          </div>
10       </>
11     )
12   }
13
14   export default Footer;
```

Footer.jsx

```
import "./Footer.css";
import emotion1 from './../assets/emotion1.png';
const Footer =()=>{
    return (
      <>
        <div className="Footer">
            <img src={emotion1} />
            <div className="Footer_div">Programmer Heejung Jang!</div>
        </div>
      </>
    )
  }

  export default Footer;
```

Footer.css파일

8253jang@daum.net
장희정

step18-client-jwt > src > components > # Footer.css > ...

```
.Footer{
    display: flex;
    justify-content: center;
    font-size: 30px;
    gap: 10px;
    padding: 30px;
    width: auto;
    background-color: rgba(0, 255, 204, 0.59);
    margin-bottom: 20px;
    border-radius: 5px;
}

.Footer> img{
    width:30px;
}

.Footer > .Footer_div{
    text-shadow:  2px 2px 8px #FF0000;
    color: white;
}
```

Footer.css

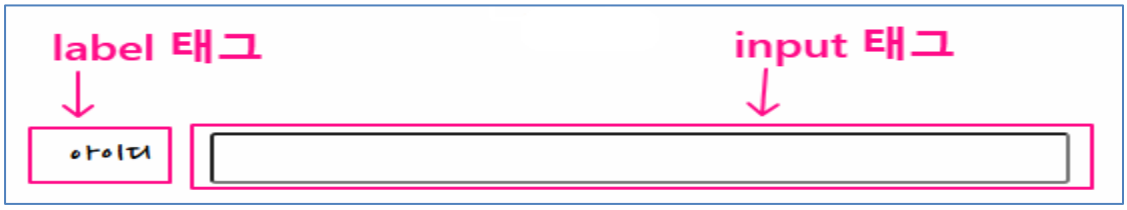
```
.Footer{
    display: flex;
    justify-content: center;
    font-size: 30px;
    gap: 10px;
    padding: 30px;
    width: auto;
    background-color: rgba(0, 255, 204, 0.59);
    margin-bottom: 20px;
    border-radius: 5px;
}

.Footer> img{
    width:30px;
}

.Footer > .Footer_div{
    text-shadow:  2px 2px 8px #FF0000;
    color: white;
}
```

8253jang@daum.net
장희정

실행화면

LabelText.jsx파일 이해하기

현재 프로젝트에서 아래의 화면이 여러 컴포넌트에서 사용되고 있다. 때문에 우리는 아래 부분만 따로 제작하여(LabelText.jsx) 필요할 때 재사용 해서 사용하도록 하자..



LabelText.jsx파일

8-client-jwt > src > components > LabelText.jsx > LabelText

```
1  import { useState } from "react";
2  import "./LabelText.css"
3
4  const LabelText =({text, name, value:initialValue})=>{
5      const [value, setValue] = useState(initialValue || "");
6      return (
7          <div className="LabelText" >
8              <label className="label">{text}</label>
9              <input className={`${value==""? 'input' : 'input_disabled' }`}
10             type={`${name=="pwd" ? "password" : "text"}`}  name={name}  value={value}
11             onChange={(e) => setValue(e.target.value)}/>
12         </div>
13     )
14 }
15
16   export default LabelText;
```

text : label에 들어갈 글씨
name : input요소가 password인지 text인지 전달
value : 초기값

vaule의 값이 없으면 className={input} 있으면 className={input_disabled} 설정

type="password" or type="text"

LabelText.jsx

```
import { useState } from "react";
import "./LabelText.css"

const LabelText =({text, name, value:initialValue})=>{
    const [value, setValue] = useState(initialValue || "");
    return (
        <div className="LabelText" >
            <label className="label">{text}</label>
            <input className={`${value==""? 'input' : 'input_disabled' }`}
            type={`${name=="pwd" ? "password" : "text"}`}  name={name}  value={value}
            onChange={(e) => setValue(e.target.value)}/>
        </div>
    )
```

```
}

  export default LabelText;
```

LabelText.css

p18-client-jwt > src > components > # LabelText.css > .LabelText .input_disabled

```
.LabelText{
    display: flex;
    gap: 10px;
   font-size:25px;
   padding: 15px 50px;
}

.LabelText .label{
  min-width: 60px;

}

.LabelText .input{
    flex: 1;
    border-radius: 3px;
    font-size:1em;
}

.LabelText .input_disabled{
    flex: 1;
    border:none;
    font-size:1em;
}
```

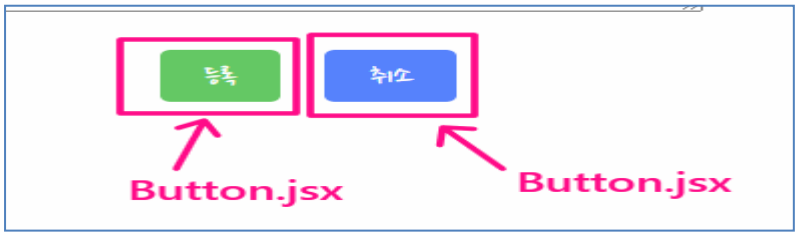
LabelText.css

```
.LabelText{
    display: flex;
    gap: 10px;
   font-size:25px;
   padding: 15px 50px;
}

.LabelText .label{
  min-width: 60px;

}

.LabelText .input{
    flex: 1;
    border-radius: 3px;
    font-size:1em;
}
```

8253jang@daum.net
장희정

```
.LabelText .input_disabled{
    flex: 1;
    border:none;
    font-size:1em;
}
```

Button.jsx파일 이해 하기



전체 프로젝트에서 button이 여러 곳에서 쓰여지고 있고 text와 css와 다르기 때문에 컴포넌트로 만들어 놓고 재 사용하자.

tep18-client-jwt > src > components > Button.jsx > default

```
1  import './Button.css';
2
3  const Button = ({text, type,  onClick})=>{
4      return (
5          <button  type={type} className={`Button Button_${type}`}
6           onClick={onClick}>{text}</button>
7      )
8  }
9
10 export default Button;
11
```

Button.jsx

```
import './Button.css';

const Button = ({text, type,   onClick})=>{
    return (
        <button   type={type} className={`Button Button_${type}`}
         onClick={onClick}>{text}</button>
    )
}

export default Button;
```

8253jang@daum.net
장희정

Button.css파일

step18-client-jwt > src > components > # Button.css > ...

```
.Button{
    background-color: rgb(236, 236, 236);
    cursor: pointer;
    border: none;
    border-radius: 5px;
    padding: 10px 20px;
    font-size: 18px;
    white-space: nowrap;/*이버튼안에 있는 text가 화면이 줄어들어도 줄바꿈되지않도록 한다.*/
}

.Button_button{
    background-color: rgb(100, 201, 100);
    color: white;
  }

  .Button_reset{
      background-color: rgb(86, 131, 253);
      color: white;
  }
```

Button.css

```
.Button{
    background-color: rgb(236, 236, 236);
    cursor: pointer;
    border: none;
    border-radius: 5px;
    padding: 10px 20px;
    font-size: 18px;
    white-space: nowrap;/*이버튼안에 있는 text가 화면이 줄어들어도 줄바꿈되지않도록 한다.*/
}

.Button_button{
    background-color: rgb(100, 201, 100);
    color: white;
  }

  .Button_reset{
      background-color: rgb(86, 131, 253);
      color: white;
  }
```

8253jang@daum.net
장희정

1)회원 가입

user > JoinForm.jsx | JoinForm.css

```
6   const JoinForm =()=>{
7       const [member, setMember] = useState({
8        id: "",
9        name: "",
10       pwd: "",
11       address: "",
12     });
13
14    // 중복체크 결과 값을 저장 할 idCheckResult
15    const [idCheckResult , setIdCheckResult] = useState(""); //중복입니다. or 사용가능합니다.
16
17   // 아이디 중복여부에 따른 css 를 적용하기 위해 상태 변수
18    const [isCheckResult , setIsCheckResult] = useState(false); //true이면 중복, false이면 사용가능
19
20     //각 text 박스에 값이 변경되었을 때
21     const changeValue = (e) => {
22
23     };
24
25
26     //가입하기
27     const submitJoin = (e)=>{
28
29
30       };
31       return (
32         <>
33         <h1 className="h1">회원가입</h1>
34
35           <div className="idDuplicate">
36             <LabelText text={"아이디"} name={"id"}   changeValue={changeValue}/>
37            <div className="idText" style={ isCheckResult ? {color: "red"} : {color: "blue" } }>{idCheckResult}</div>
38           </div>
39
40           <LabelText text={"이름"} name={"name"}   changeValue={changeValue} />
41           <LabelText text={"비밀번호"} name={"pwd"}    changeValue={changeValue}/>
42           <LabelText text={"주소"} name={"address"}   changeValue={changeValue} />
43
44           <div className="divBtn">
45               <Button text={"회원가입"}  type={"button"}  onClick={submitJoin} />
46               <Button text={"취소"}  type={"reset"} />
47           </div>
48         </>
49       )
50     }
51
52     export default JoinForm;
```

JoinForm.jsx

```
import { useState } from "react";
import Button from "../../components/Button";

import LabelText from "../../components/LabelText";
import "./JoinForm.css";
const JoinForm =()=>{

    const [member, setMember] = useState({
```

```
    id: "",
    name: "",
    pwd: "",
    address: "",
  });

 // 중복체크 결과 값을 저장 할 idCheckResult
 const [idCheckResult , setIdCheckResult] = useState(""); //중복입니다. or 사용가능합니다.

// 아이디 중복여부에 따른 css 를 적용하기 위해 상태 변수
 const [isCheckResult , setIsCheckResult] = useState(false); //true이면 중복, false이면 사용가능

  //각 text 박스에 값이 변경되었을 때
  const changeValue = (e) => {

  };

  //가입하기
  const submitJoin = (e)=>{


    };
    return (
      <>
      <h1 className="h1">회원가입</h1>

        <div className="idDuplicate">
          <LabelText text={"아이디"} name={"id"}     changeValue={changeValue}/>
         <div className="idText" style={ isCheckResult ? {color: "red"} : {color: "blue" } }>{idCheckResult}</div>
        </div>

        <LabelText text={"이름"} name={"name"}     changeValue={changeValue} />
        <LabelText text={"비밀번호"} name={"pwd"}      changeValue={changeValue}/>
        <LabelText text={"주소"} name={"address"}     changeValue={changeValue} />

        <div className="divBtn">
            <Button text={"회원가입"}   type={"button"}   onClick={submitJoin} />
            <Button text={"취소"}   type={"reset"} />
        </div>
      </>
    )
  }

  export default JoinForm;
```

JoinForm.css 파일

8253jang@daum.net
장희정

step18-client-jwt > src > pages > user > # JoinForm.css > ...

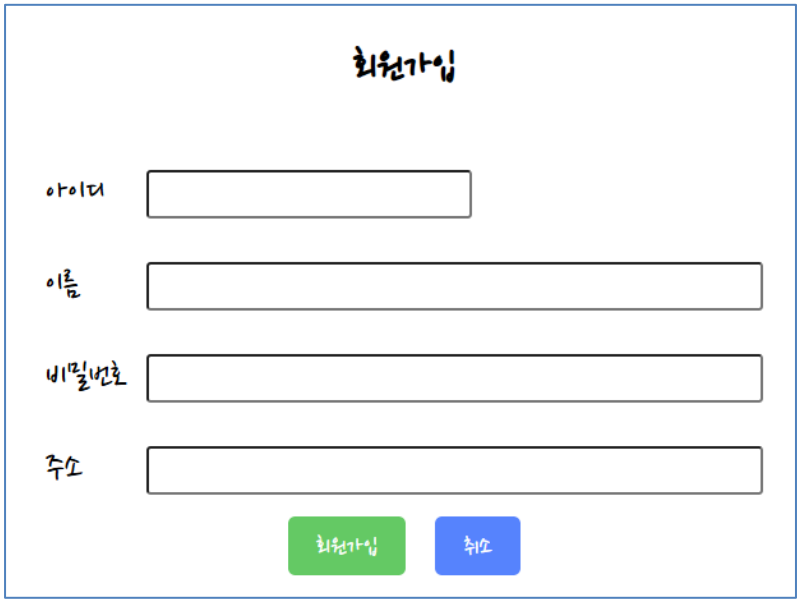
```
1
2   .idDuplicate{
3       display: flex;
4       justify-content:flex-start;
5
6   }
7
8   .idDuplicate .idText{
9      flex: 1;
10     padding-top: 20px ;
11  }
12  .divBtn{
13      display: flex;
14      gap: 20px;
15      justify-content: center;
16  }
17
18  .h1{
19      text-align: center;
20      font-size: 30px;
21      padding: 20px;
22  }
23
24  .idText{
25      color: red;
26      font-weight: bold;
27      font-size: 1.3em;
28  }
```

JoinForm.css

```
.idDuplicate{
    display: flex;
    justify-content:flex-start;

}

.idDuplicate .idText{
   flex: 1;
   padding-top: 20px ;
}
.divBtn{
    display: flex;
    gap: 20px;
    justify-content: center;
}

.h1{
    text-align: center;
    font-size: 30px;
    padding: 20px;
}

.idText{
    color: red;
```

```
    font-weight: bold;
    font-size: 1.3em;
}
```

실행화면



2) 로그인

user >LoginForm.jsx | LoginForm.css

8253jang@daum.net
장희정

```
1   import Button from "../../components/Button";
2   import LabelText from "../../components/LabelText";
3   import  { useState } from "react";
4   import "./LoginForm.css";
5
6   const LoginForm =()=>{
7
8     // 인증에 필요한 username, password 상태관리를 위한 useState
9     const [member, setMember] = useState({
10      username: "",
11      password: "",
12    });
13
14    // input 에 값이 입력될 때 상태 값 수정
15    const changeValue = (e) => {
16      setMember({
17        ...member,
18        [e.target.name]: e.target.value,
19      });
20    }; //
21
22   //로그인
23    const submitLogin = (e) => {
24
25    }
26
27    //취소
28     const cancelBack=()=>{
29
30     }
33     return (
34       <div className="LoginForm">
35         <h1 className="h1">로그인</h1>
36         <LabelText text={"아이디"} name={"username"}  changeValue={changeValue} />
37         <LabelText text={"비밀번호"} name={"password"} changeValue={changeValue} />
38
39         <div className="divBtn">
40             <Button text={"로그인"}  type={"button"}  onClick={submitLogin}/>
41             <Button text={"취소"}  type={"reset"}  onClick={cancelBack}/>
42         </div>
43
44       </div>
45     )
46   }
47
48   export default LoginForm;
```

LoginForm.jsx

```
import Button from "../../components/Button";
import LabelText from "../../components/LabelText";
import  { useState } from "react";
import "./LoginForm.css";
```

```
const LoginForm =()=>{

  // 인증에 필요한 username, password 상태관리를 위한 useState
  const [member, setMember] = useState({
     username: "",
     password: "",
  });

  // input 에 값이 입력될 때 상태 값 수정
  const changeValue = (e) => {
     setMember({
        ...member,
        [e.target.name]: e.target.value,
     });
  }; //

 //로그인
  const submitLogin = (e) => {

  }

  //취소
   const cancelBack=()=>{

   }

     return (
        <div className="LoginForm">
           <h1 className="h1">로그인</h1>
           <LabelText text={"아이디"} name={"username"}   changeValue={changeValue} />
           <LabelText text={"비밀번호"} name={"password"} changeValue={changeValue} />

           <div className="divBtn">
                <Button text={"로그인"}   type={"button"}   onClick={submitLogin}/>
                <Button text={"취소"}   type={"reset"}   onClick={cancelBack}/>
           </div>

        </div>
     )
  }

  export default LoginForm;
```

LoginForm.css

8253jang@daum.net
장희정

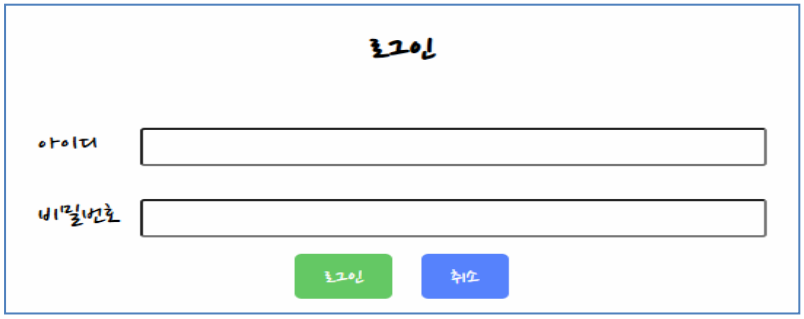
step18-client-jwt > src > pages > user > # LoginForm.css > .LoginForm

```
.LoginForm{

}

.LoginForm .h1{
    text-align: center;
    font-size: 30px;
    padding: 20px;
}
```

LoginForm.css

```
.LoginForm{

}

.LoginForm .h1{
    text-align: center;
    font-size: 30px;
    padding: 20px;
}
```

실행화면



1) 게시물 등록

board > SaveForm.jsx | SaveForm.css

SaveForm.jsx파일

```
1   import { useState } from "react";
2   import Button from "../../components/Button";
3   import LabelText from "../../components/LabelText";
4   import "./SaveForm.css";
5
6   const SaveForm =()=>{
7
8     //게시물 상태관리
9     const [board, setBoard] = useState({
10      title: "",
11      content: "",
12      memberNo: localStorage.getItem("memberNo"),
13    }); //제목, 작성자=memberNo, 내용
14
15    //input 에 값이 변경될 때
16    const changeValue = (e) => {
17      setBoard({
18        ...board,
19        [e.target.name]: e.target.value,
20      });
21    };
22
23    //등록하기 클릭
24    const submitBoard = (e) => {
25
26    };
28    return (
29      <>
30      <h1 className="h1">  게시물 등록하기 </h1>
31
32        <LabelText text={"title"} name={"title"} changeValue={changeValue}/>
33        <LabelText text={"writer"} name={"name"} readOnly={"yes"}/>
34
35        <div className="divContent">
36          <div className="divContent_label">cotent</div>
37          <textarea className="divContent_textarea" placeholder="내용을 입력해주세요^^"  name="content" onChange={changeValue}></textarea>
38       </div>
39
40        <div className="divBtn">
41          <Button text={"등록"}  type={"button"}  onClick={submitBoard}/>
42          <Button text={"취소"}  type={"reset"}  onClick={()=>navigator(-1)}/>
43        </div>
44      </>
45    )
46  }
47
48  export default SaveForm;
```

SaveForm.jsx

```
import { useState } from "react";
import Button from "../../components/Button";
import LabelText from "../../components/LabelText";
import "./SaveForm.css";

const SaveForm =()=>{

  //게시물 상태관리
  const [board, setBoard] = useState({
    title: "",
    content: "",
```

```
    memberNo: localStorage.getItem("memberNo"),
  }); //제목, 작성자=memberNo, 내용

  //input 에 값이 변경될 때
  const changeValue = (e) => {
    setBoard({
      ...board,
      [e.target.name]: e.target.value,
    });
  };

  //등록하기 클릭
  const submitBoard = (e) => {

  };

    return (
      <>
      <h1 className="h1">  게시물 등록하기 </h1>

        <LabelText text={"title"} name={"title"} changeValue={changeValue}/>
        <LabelText text={"writer"} name={"name"} readOnly={"yes"}/>

        <div className="divContent">
          <div className="divContent_label">cotent</div>
                <textarea  className="divContent_textarea"  placeholder="내용을  입력해주세요^^"    name="content"
onChange={changeValue}></textarea>
        </div>

        <div className="divBtn">
            <Button text={"등록"}  type={"button"}  onClick={submitBoard}/>
            <Button text={"취소"}  type={"reset"}  onClick={()=>navigator(-1)}/>
        </div>
      </>
    )
  }

  export default SaveForm;
```

SaveForm.css파일

8253jang@daum.net
장희정

```
1
2   .idDuplicate{
3       display: flex;
4       justify-content:flex-start;
5   }
6
7   .idDuplicate .idText{
8     flex: 1;
9     padding-top: 20px ;
10  }
11  .divBtn{
12      display: flex;
13      gap: 20px;
14      justify-content: center;
15  }
16
17  .h1{
18      text-align: center;
19      font-size: 30px;
20      padding: 20px;
21  }
22
23
24  .divContent{
25    display: flex;
26    gap: 10px;
27    margin-bottom: 30px;
28  }
29
30  .divContent .divContent_label{
31      font-size: 25px ;
32      margin-left: 50px;
33  }
34
35  .divContent .divContent_textarea{
36      padding: 20px;
37      width:60%;
38      min-height: 200px;
39      resize:vertical;
40      box-sizing: border-box;
41      font-size: 25px ;
42  }
```

SaveForm.css

```
.idDuplicate{
    display: flex;
    justify-content:flex-start;
}

.idDuplicate .idText{
  flex: 1;
```

```
    padding-top: 20px ;
}
.divBtn{
      display: flex;
      gap: 20px;
      justify-content: center;
}

.h1{
      text-align: center;
      font-size: 30px;
      padding: 20px;
}

.divContent{
  display: flex;
  gap: 10px;
  margin-bottom: 30px;
}

.divContent .divContent_label{
      font-size: 25px ;
      margin-left: 50px;
}

.divContent .divContent_textarea{
      padding: 20px;
      width:60%;
      min-height: 200px;
      resize:vertical;
      box-sizing: border-box;
      font-size: 25px ;
}
```

실행화면

8253jang@daum.net
장희정

2) 전체게시물

components > BoardItem.jsx | BoardItem.css

board > Home.jsx | Home.css

BoardItem.jsx

```
step18-client-jwt > src > components > BoardItem.jsx > BoardItem
1  import { Link } from "react-router-dom";
2  import "./BoardItem.css";
3  const BoardItem =({title , writer})=>{
4    return (
5      <div className="BoardItem">
6        <div className="title_writer">제목 : {title} /작성자 : ({writer})</div>
7        <div className="detailLink">
8          <Link to={"/board/1"} >상세보기 </Link>
9        </div>
10     </div>
11   )
12 }
13
14 export default BoardItem;
```

BoardItem.jsx

```
import { Link } from "react-router-dom";
import "./BoardItem.css";
const BoardItem =({title , writer})=>{
  return (
    <div className="BoardItem">
      <div className="title_writer">제목 : {title} /작성자 : ({writer})</div>
      <div className="detailLink">
```

```
            <Link to={"/board/1"} >상세보기  </Link>
        </div>
      </div>
   )
}

export default BoardItem;
```

BoardItem.css파일

step18-client-jwt > src > components > # BoardItem.css > .BoardItem .title_writer

```
.BoardItem{
    border: 1px solid gainsboro;
    border-radius: 5px;
    padding: 10px;
}

.BoardItem .title_writer{
   font-size: 25px;
}

.BoardItem  .detailLink{
    margin-top: 10px;
    padding: 10px;
    font-size: 20px;
    border-radius: 15px;
    background-color: #FFFF00;
    width: 100px;
    text-align: center;
}

.BoardItem  .detailLink > a{
    color: blue;
}
```

BoardItem.css

```
.BoardItem{
    border: 1px solid gainsboro;
    border-radius: 5px;
    padding: 10px;
}

.BoardItem .title_writer{
   font-size: 25px;
}

.BoardItem   .detailLink{
```

```
    margin-top: 10px;
    padding: 10px;
    font-size: 20px;
    border-radius: 15px;
    background-color: #FFFF00;
    width: 100px;
    text-align: center;
}

.BoardItem   .detailLink > a{
    color: blue;
}
```

Home.jsx파일

step18-client-jwt > src > pages > board > Home.jsx > default

```
import BoardItem from "../../components/BoardItem";
import "./Home.css";

const Home =()=>{
    return (
      <div className="Home">
        <h1>전체게시물 LIST</h1>
        <BoardItem title={"Spring1"} writer={"장희정"}/>
        <BoardItem title={"Spring1"} writer={"장희정"}/>
        <BoardItem title={"Spring1"} writer={"장희정"}/>
        <BoardItem title={"Spring1"} writer={"장희정"}/>
      </div>
    )
  }

  export default Home;
```

Home.css 파일

step18-client-jwt > src > pages > board > Home.css > .Home h1

```
.Home{
    padding: 20px;
}

.Home h1{
  text-align: center;
}

```

Home.css

```
.Home{
```

```
    padding: 20px;
}

.Home h1{
  text-align: center;
}
```

실행화면

3) 상세보기

board > Detail.jsx | Detail.css

Details.jsx파일

8253jang@daum.net
장희정

```
import Button from "../../components/Button";
import LabelText from "../../components/LabelText";
import "./Detail.css";
import { useState } from "react";

const Detail =()=>{

    const [board, setBoard] = useState({
      id: "",
      title: "",
      content: "",
      member: {},
      regDate:""
    });


    //삭제
    const deleteBoard = () => {

  };//삭제끝

  //수정 클릭
  const updateBoard = () => {


  };
  return (
    <div className="Detail">
     <h1>상세보기</h1>

      <LabelText text={"글번호"}  initValue={board.id}/>
      <LabelText text={"제목"}  initValue={board.title}/>
      <LabelText text={"작성자"}  initValue={board.member.name}/>
      <LabelText text={"등록일"}  initValue={board.regDate}/>

      <div className="divContent">
        <div className="divContent_label">cotent</div>
        <textarea className="divContent_textarea" placeholder="오늘은 어땠나요?"
         name="content"  disabled value={board.content}></textarea>
    </div>

    { board.member.name === localStorage.getItem("name") &&
     <div className="divBtn">
          <Button text={"수정"}  type={"button"}  onClick={updateBoard}/>
          <Button text={"삭제"}  type={"button"} onClick={deleteBoard} />
     </div>
    }

    </div>
  )
}

export default Detail;
```

Detail.jsx

```
import Button from "../../components/Button";
import LabelText from "../../components/LabelText";
import "./Detail.css";
import { useState } from "react";

const Detail =()=>{

  const [board, setBoard] = useState({
    id: "",
    title: "",
    content: "",
    member: {},
    regDate:""
  });

  //삭제
  const deleteBoard = () => {

 };//삭제끝

 //수정 클릭
 const updateBoard = () => {



 };

   return (
     <div className="Detail">
      <h1>상세보기</h1>

       <LabelText text={"글번호"}  initValue={board.id}/>
       <LabelText text={"제목"}  initValue={board.title}/>
       <LabelText text={"작성자"}  initValue={board.member.name}/>
       <LabelText text={"등록일"}  initValue={board.regDate}/>

       <div className="divContent">
         <div className="divContent_label">cotent</div>
         <textarea className="divContent_textarea" placeholder="오늘은 어땠나요?"
          name="content"  disabled value={board.content}></textarea>
      </div>

     { board.member.name === localStorage.getItem("name") &&
      <div className="divBtn">
           <Button text={"수정"}  type={"button"}  onClick={updateBoard}/>
           <Button text={"삭제"}  type={"button"} onClick={deleteBoard} />
       </div>
```

```
      }

      </div>
    )
  }

  export default Detail;
```

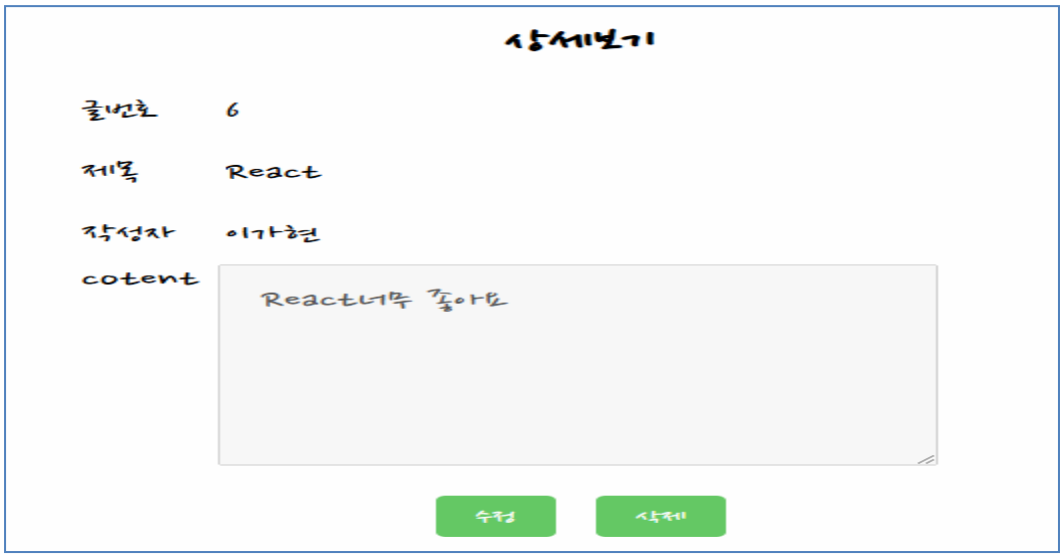
Detail.css 파일

```
p18-client-jwt > src > pages > board > # Detail.css > .Detail h1
1  .Detail h1{
2    text-align: center;
3  }
4
5  .Detail > input{
6      border: none;
7  }
```

Detail.css

```
.Detail h1{
  text-align: center;
}

.Detail > input{
    border: none;
}
```

실행화면



8253jang@daum.net
장희정

4) 수정하기

board > UpdateForm.jsx | UpdateForm.css

UpdateForm.jsx파일

```
import { useNavigate, useParams } from "react-router-dom";
import Button from "../../components/Button";
import LabelText from "../../components/LabelText";
import './UpdateForm.css';
import { useEffect, useState } from "react";

const UpdateForm =()=>{

  const { id } = useParams(); //

  const [board, setBoard] = useState({
    title: "",
    content: "",
    member: {},
  });

  //페이지 이동하는 방법
  const navigator = useNavigate();

  useEffect(() => {

  }, []);


  const changeValue = (e)=>{
      setBoard(
        {...board ,
         [e.target.name] : e.target.value
      })
  }

 const submitBoard = (e)=>{

 };
```

```
35      return (
36        <>
37        <h1 className="h1">  수정하기 </h1>
38        <form>
39
40          <LabelText text={"title"} name={"title"} initValue={board.title}
41          changeValue={changeValue} />
42
43          <LabelText text={"writer"} name={"name"}  readOnly="yes"/>
44
45          <div className="divContent">
46            <div className="divContent_label">cotent</div>
47            <textarea
48            className="divContent_textarea"
49            placeholder="내용을 입력해주세요"
50             name="content"
51             value={board.content}
52             onChange={changeValue}></textarea>
53         </div>
54
55          <div className="divBtn">
56              <Button text={"수정"}  type={"button"}  onClick={submitBoard}/>
57              <Button text={"취소"}  type={"reset"} />
58          </div>
59          </form>
60        </>
61      )
62    }
63
64    export default UpdateForm;
```

UpdateForm.jsx

```
import { useNavigate, useParams } from "react-router-dom";
import Button from "../../components/Button";
import LabelText from "../../components/LabelText";
import './UpdateForm.css';
import { useEffect, useState } from "react";

const UpdateForm =()=>{

  const { id } = useParams(); //

  const [board, setBoard] = useState({
    title: "",
    content: "",
    member: {},
  });

  //페이지 이동하는 방법
  const navigator = useNavigate();
```

```
 useEffect(() => {

 }, []);


 const changeValue = (e)=>{
     setBoard(
         {...board ,
          [e.target.name] : e.target.value
     })
 }

const submitBoard = (e)=>{

};
    return (
      <>
       <h1 className="h1">   수정하기  </h1>
      <form>

         <LabelText text={"title"} name={"title"} initValue={board.title}
         changeValue={changeValue} />

         <LabelText text={"writer"} name={"name"}   readOnly="yes"/>

         <div className="divContent">
            <div className="divContent_label">cotent</div>
            <textarea
            className="divContent_textarea"
            placeholder="내용을  입력해주세요"
             name="content"
             value={board.content}
             onChange={changeValue}></textarea>
        </div>

         <div className="divBtn">
              <Button text={"수정"}   type={"button"}   onClick={submitBoard}/>
              <Button text={"취소"}   type={"reset"} />
         </div>
         </form>
      </>
    )
 }

 export default UpdateForm;
```

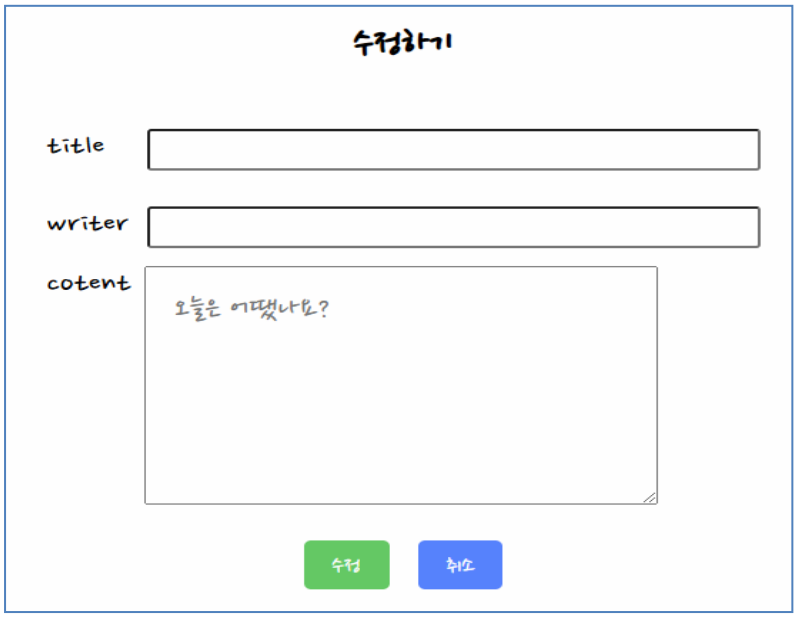
8253jang@daum.net
장희정

UpdateForm.css파일

```
.idDuplicate{
    display: flex;
    justify-content:flex-start;
}

.idDuplicate .idText{
   flex: 1;
   padding-top: 20px ;
}
.divBtn{
    display: flex;
    gap: 20px;
    justify-content: center;
}

.h1{
    text-align: center;
    font-size: 30px;
    padding: 20px;
}

.divContent{
  display: flex;
  gap: 10px;
  margin-bottom: 30px;
}

.divContent .divContent_label{
    font-size: 25px ;
    margin-left: 50px;
}

.divContent .divContent_textarea{
    padding: 20px;
    width:60%;
    min-height: 200px;
    resize:vertical;
    box-sizing: border-box;
    font-size: 25px ;
}
```

UpdateForm.css

```
.idDuplicate{
    display: flex;
    justify-content:flex-start;
}

.idDuplicate .idText{
   flex: 1;
   padding-top: 20px ;
}
.divBtn{
    display: flex;
    gap: 20px;
    justify-content: center;
}

.h1{
```

```
    text-align: center;
    font-size: 30px;
    padding: 20px;
}

.divContent{
  display: flex;
  gap: 10px;
  margin-bottom: 30px;
}

.divContent .divContent_label{
    font-size: 25px ;
    margin-left: 50px;
}

.divContent .divContent_textarea{
    padding: 20px;
    width:60%;
    min-height: 200px;
    resize:vertical;
    box-sizing: border-box;
    font-size: 25px ;
}
```

실행화면